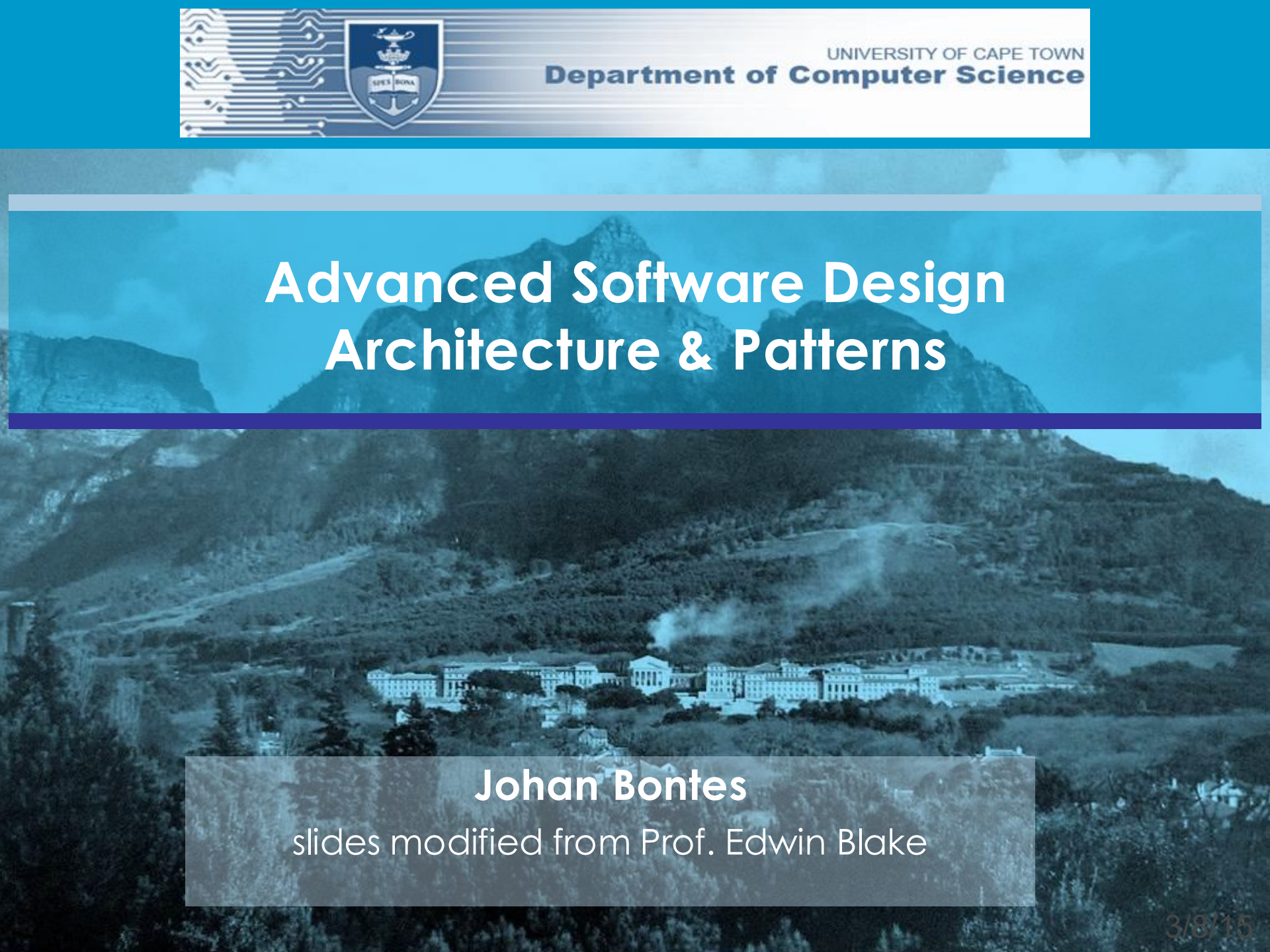


Advanced Software Design Architecture & Patterns

Johan Bontes

slides modified from Prof. Edwin Blake

ARCHITECTURE CENTRED APPROACH

Architecture Centred Approach

Introducing Patterns

Key Architecture Patterns

Model-View-Controller

Layered Architecture

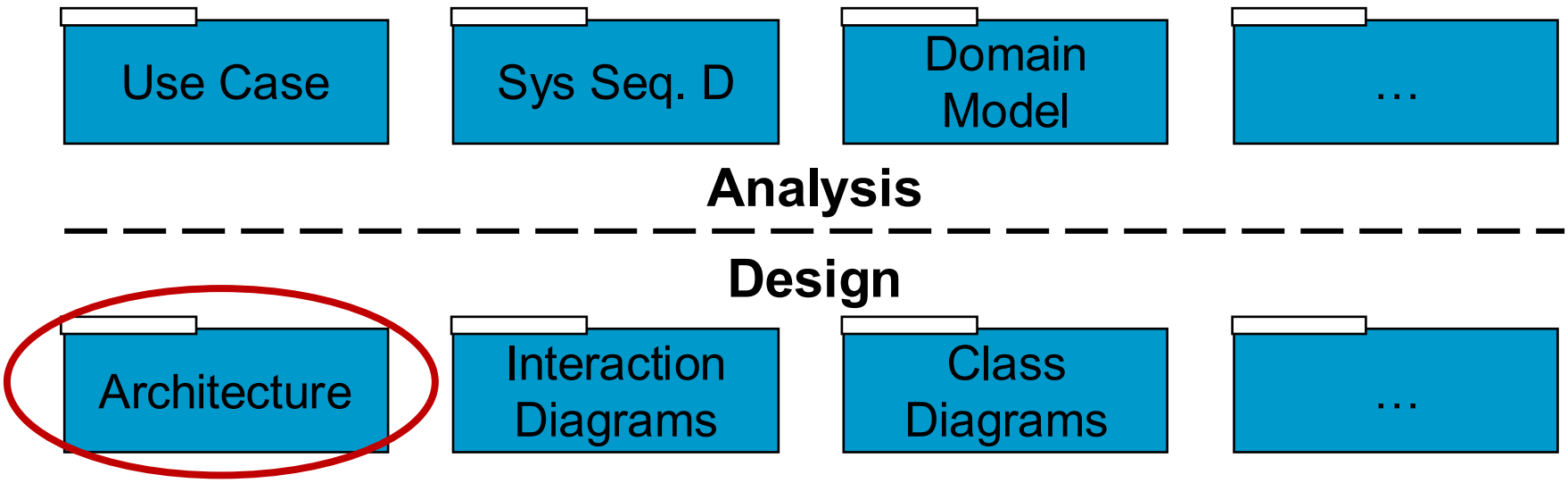
Patterns and Concurrency

Conclusion

Architecture centred approach

- We place an emphasis on design
  - Design pervades engineering activities
  - Important aspect of Computer Science
- An incorrect interpretation of the lean documentation approach of Agile development leads to inadequate levels of architectural design information
- Architecture Decisions are system-wide Strategic Decisions
  - Use Object-Orientation
  - Choose overall Architectural pattern (see later)
  - Choice of API
- Local detailed tactical decisions are use of design patterns, refactoring decisions, …

Requirements and analysis leads to design



1. we are investigators learning about a problem
2. change roles: now designers making working solution
3. goto 1

What is a software architecture then?

- Software architecture ⇒ *large scale*
  - big ideas
    - motivations
    - constraints
    - organization
    - patterns
    - responsibilities
    - connections
  - Very interested in the non-functional requirements
    - ensure that these can be met
    - or if cannot be met, suggest trade-offs...
    - For example: trends, cost, maintainability.

INTRODUCING PATTERNS

Software Architecture
Introducing Patterns
Key Architecture Patterns
Model-View-Controller
Layered Architecture
Patterns and Concurrency
Conclusion

Introducing architecture

- Think about your own programming style.
  - A large portion of it is most likely the way you structure your solutions, rather than the minute details of your code.
  - If you could reuse the structure from one project to the next, you would gain all the benefits of reuse.
- IT designer community has a growing body of past computer solutions for ideas and inspiration.
  - little progress on documenting experiences and knowledge
- Best described using architectural templates

Patterns are encapsulated experience

- Software architecture not from empty slate.
- New design based on previous “similar” designs.
- Architectural Patterns attempt to guide a new design with insight into typical problems
- Attempt to encapsulate the basic approaches for similar types of problems.
- Just like accountancy:
  every factory: same procedures, same blueprint

Aims of architecture patterns

- Enhance reusability of object-oriented code
  - Well-structured object-oriented systems have recurring patterns of classes and objects
  - Knowledge of the patterns that have worked in the past allows:
    - a designer to be more productive, and
    - the resulting designs to be more flexible and reusable.

Benefits of all architectural patterns

- Capture expertise and communicate to non-experts in an encapsulated design pattern
- Common language amongst developers
  - Improve design understandability
- Easier to reuse good designs, avoid non-reusable alternatives
- Facilitate design modifications
  - The design is more easily understood
- Improve design documentation
  - Start system documentation with UML design pattern

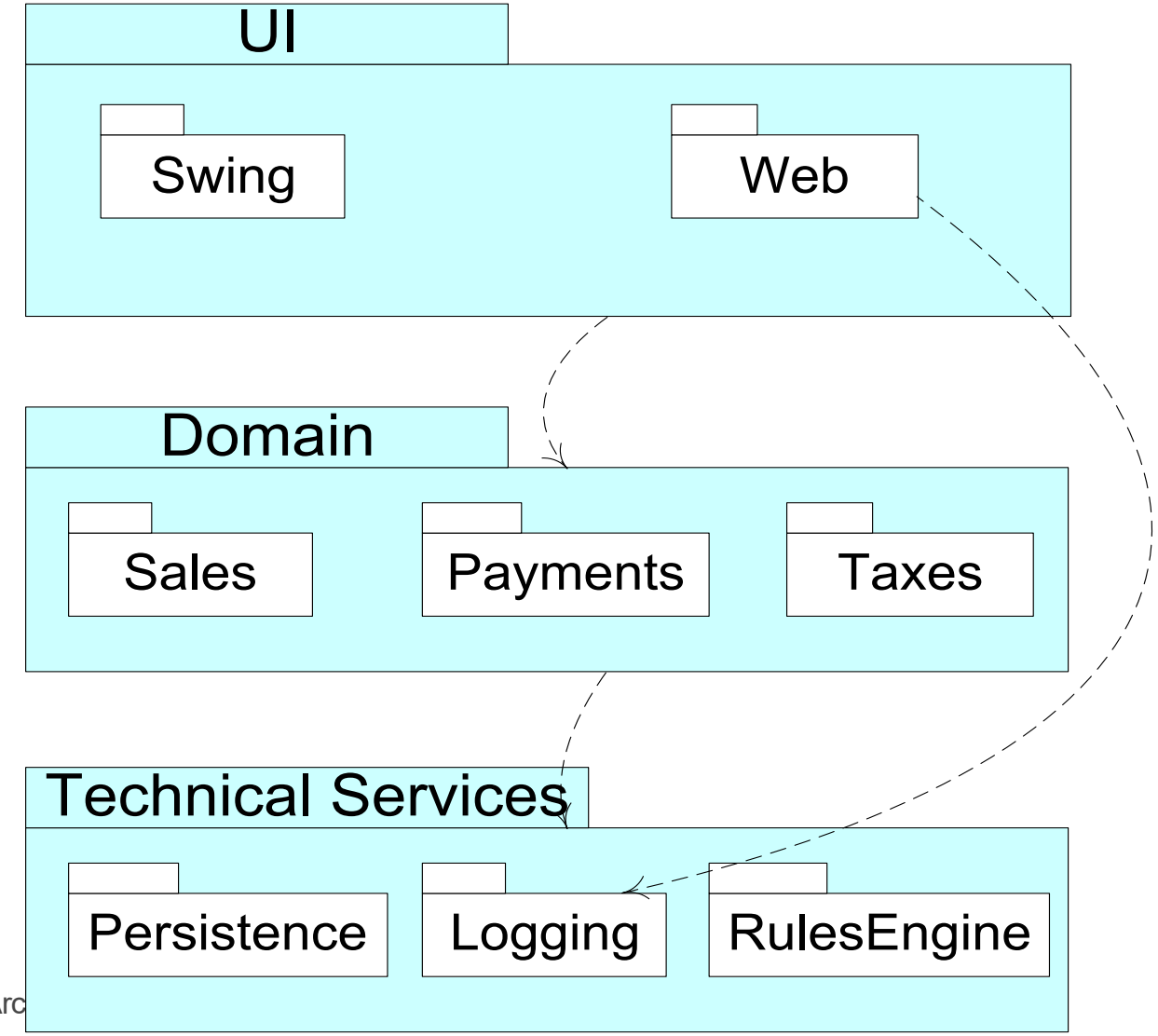
KEY ARCHITECTURE PATTERNS

Software Architecture
Introducing Patterns
Key Architecture Patterns
Model-View-Controller
Layered Architecture
Patterns and Concurrency
Conclusion

Layered Architecture Pattern

Name	Layered architecture
Problem	Building new facilities on top of existing systems, or Development spread across several teams, one team one layer, or Need for multi-level security.
Solution	Organise system into layers with related functionality associated with each layer. A layer provides services to layers above it. Lowest-level layers are core services likely to be used throughout the system.
Pro	Can easily swap one layer for another with the same interface. Redundant facilities (e.g.: authentication) can be provided in each layer to increase the dependability of the system.
Con	In practice, providing clean separation between layers is often difficult. High-level layers may have to interact directly with lower-level layers rather than through intermediate layers. Can be slow due to multiple levels between high and low levels.

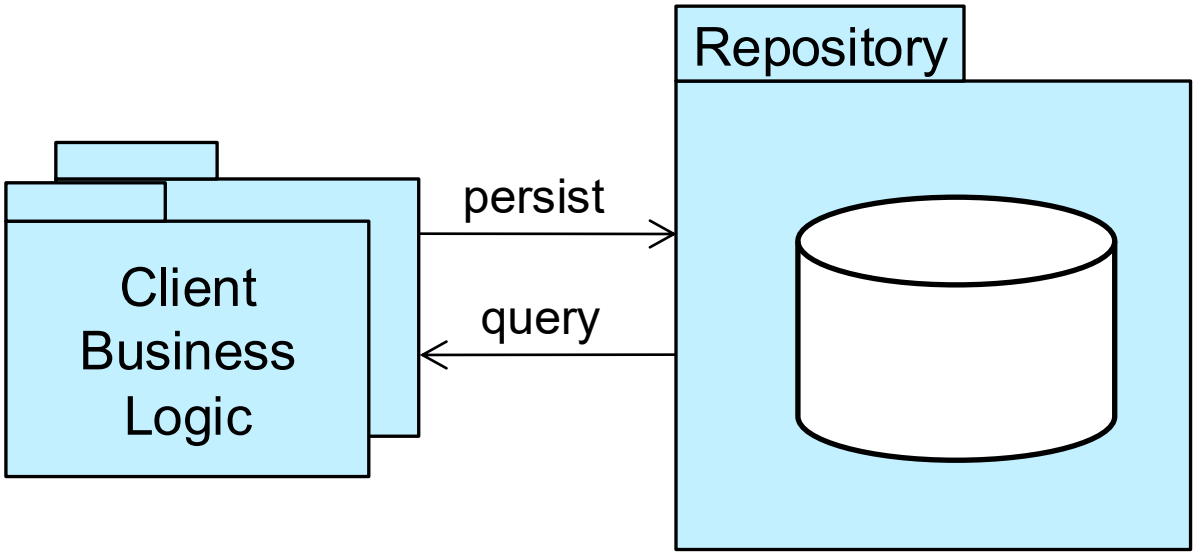
Layered Architecture Diagram



Repository Pattern

Name	Repository
Problem	Large volumes of information need to be stored for a long time, or Data-driven systems where adding data triggers an action or tool.
Solution	Store all data in central repository accessible to all components. Components only interact through repository.
Pro	Components can be independent – no need to interact with other components. Changes by one component can be propagated to all components. All data is managed in one place (e.g., backups done at the same time).
Con	Repository = “single point of failure”. Problem in repository affects the whole system. Routing all traffic through repo may be slow. Distributing repo across multiple computers may be complex.

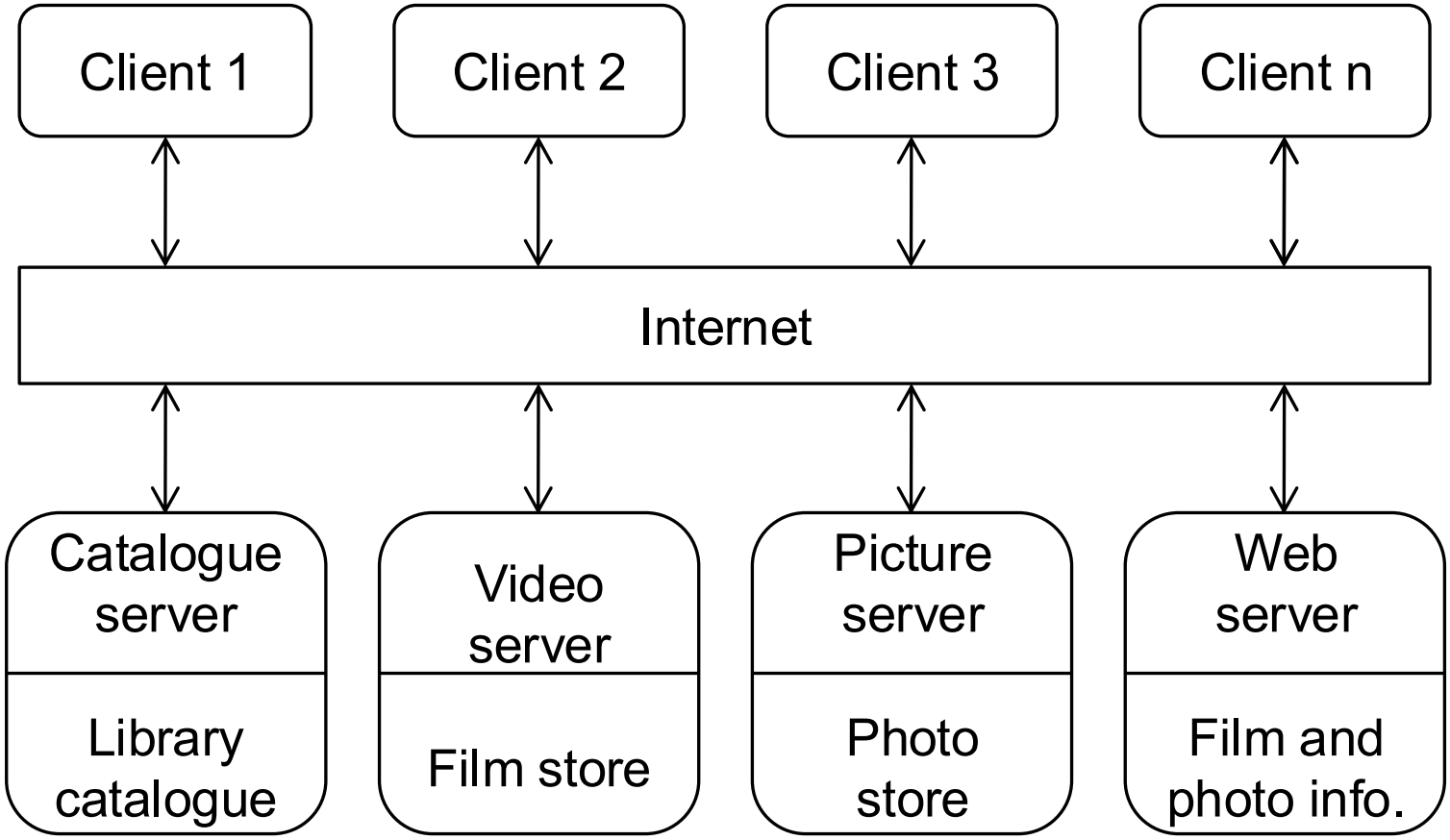
Repository Diagram



Client–Server Pattern

Name	Client-server
Problem	Data in shared database must be accessed from multiple locations, or Servers must be replicatable, or Load on a system is variable.
Solution	Functionality of system is organized into services. Each service on separate server. Clients use these services and access servers use them.
Pro	Servers can be distributed across a network. Servers can added or upgraded with minimal disruption. General functionality (e.g., a printing service) is available to all clients clients do not implement functionality.
Con	Each service = “single point of failure”: risk of denial of service attacks or server failure. Performance may be unpredictable due to network or system. Management problems if servers owned by different organizations.

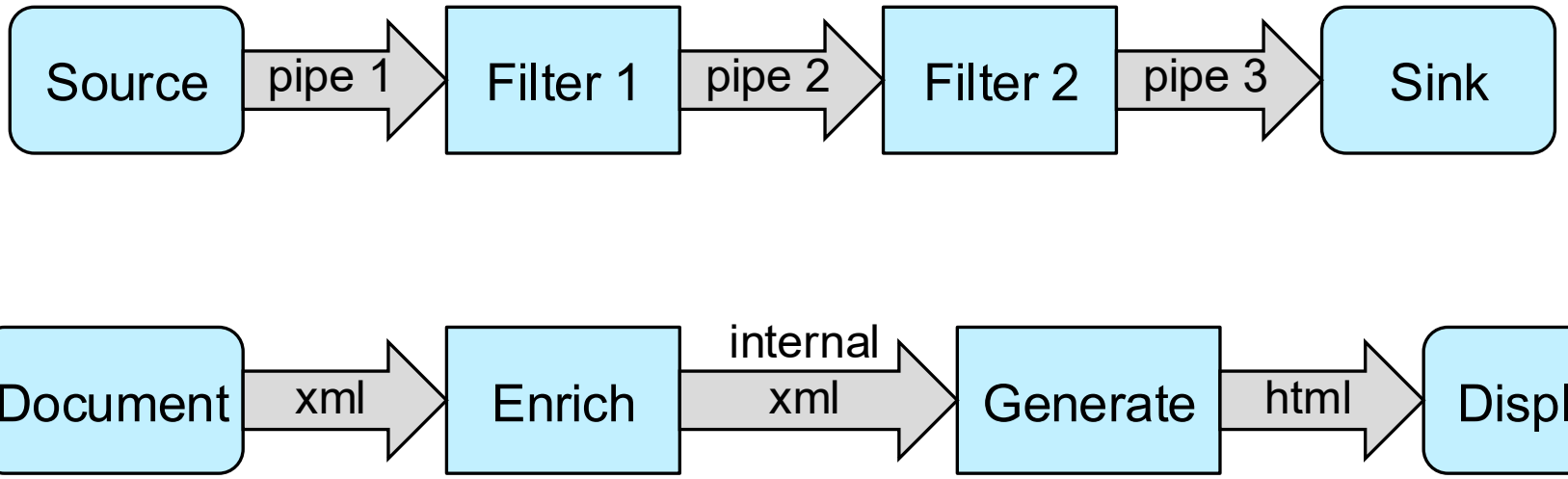
Client–Server Diagram for a Film Library



Pipe and Filter Pattern

Name	Pipe and filter
Problem	Used for data processing (both batch- and transaction-based) where inputs are processed in stages to generate related outputs.
Solution	Each processing component (filter) is discrete and carries out one type of data transformation. The data flows (as in a pipe) from one component to another.
Pro	Easy to understand and supports transformation reuse. Workflow style matches structure of many business processes. Adding transformations is simple. Can be either a sequential or concurrent system.
Con	Data transfer format must be standardised between transformations. Each transformer decodes input and encodes output to agreed form. Increased system overhead. Impossible to reuse transformations if data structures incompatible.

Pipes and Filters to Break Processing into Steps



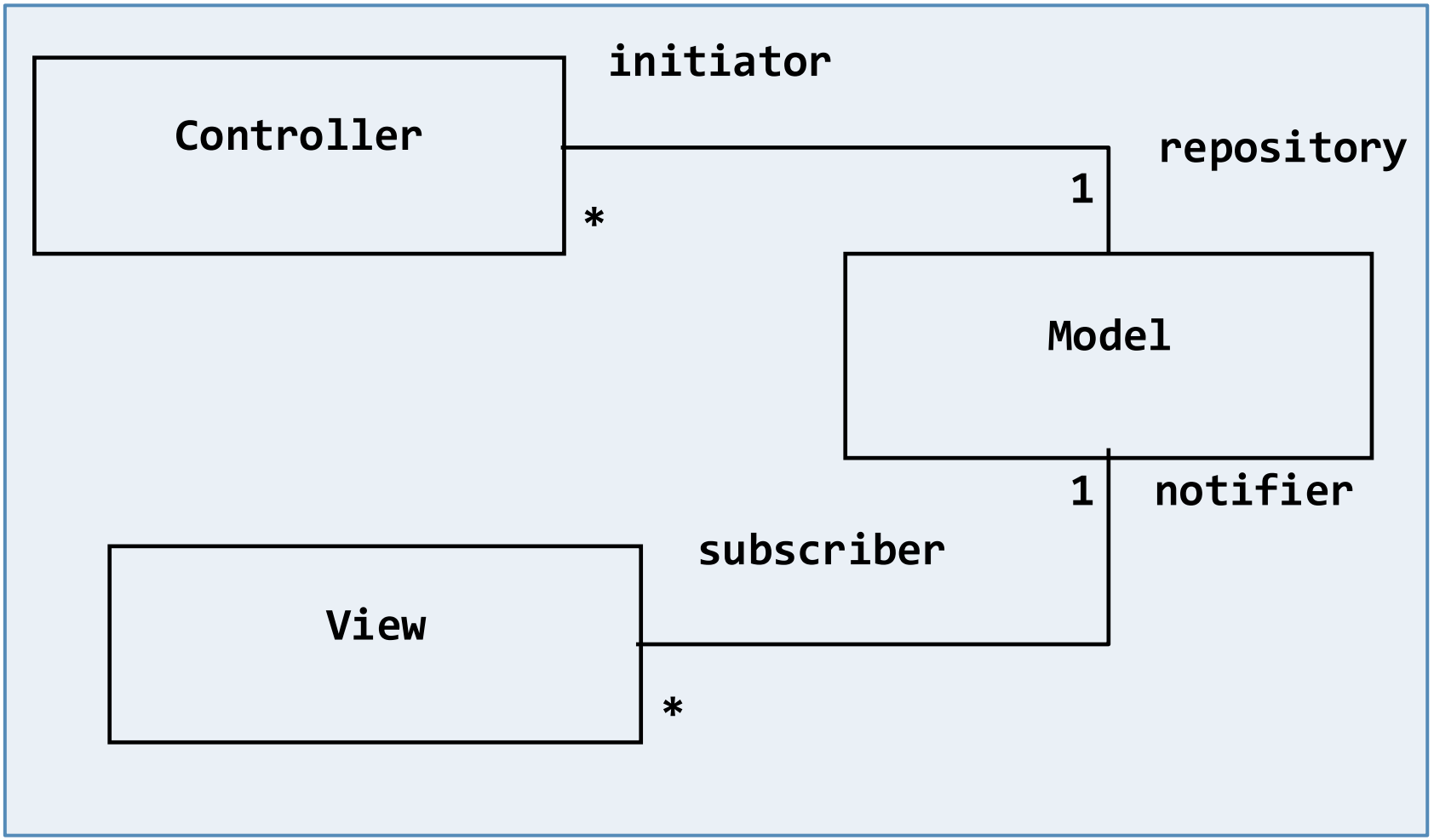
Key aim of these patterns for architecture

- Cohesion
  - Communication within the module
- Coupling
  - Communication between modules
- Minimise coupling but maximise cohesion

Model-View-Controller (MVC) Pattern

Name	MVC (Model-View-Controller)
Problem	Multiple ways to view and interact with data. Or if future interaction and presentation of data is unknown.
Solution	Separate presentation + interaction from system data. The system is structured into 3 logical interacting components. Model component manages the system data + operations. View component defines and manages presentation. Controller component manages user interaction *(e.g., key presses, mouse clicks, etc.)* passes interactions to View and Model.
Pro	Decouples data from representation. Same data can be shown in different ways.
Con	Additional code and complexity if simple data model + interactions

Model-View-Controller (MVC) Triad



Model	representation of the application
View	way information is displayed
Controller	mapping from user actions to operations on model

Model/View/Controller

- Subsystems are classified into 3 different types
  - Model: application domain knowledge
  - View: display domain objects to the user
  - Controller: interaction with the user and notifying views of changes in the model.

- MVC: special case of repository architecture:
  - Model implements central data store, Controller subsystem explicitly dictates control flow

Components of MVC: Model

- model carries out some task
- no concern for “look and feel” presented to the user
- purely functional interface,
  has a set of public functions for all its functionality
- query methods: get current model state
- mutator methods: modify state

Components of MVC: Model

- Model can “register” views and “notifies” registered views when state is changed.
- Java: Model = class extending `java.util.Observable`
- Superclass provides register/notify functionality

https://docs.oracle.com/javase/8/docs/api/java/util/Observable.html

Components of MVC: View

- View provides GUI for a model.
  Displays by querying its Model.

- When a user manipulates View, it informs Controller

- Java: Views built using AWT or SWING.
  Views implement `java.util.Observer` interface.

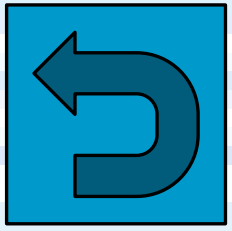
https://docs.oracle.com/javase/8/docs/api/java/util/Observer.html

Components of MVC: Controller

- Views link to multiple controllers.
  Controller updates Model as needed when user interacts with linked View.
- Controller calls Model mutator methods to update its state.
- Then Model notifies ALL registered Views of changes and they update display to the user.
- Java: Controllers are event listeners

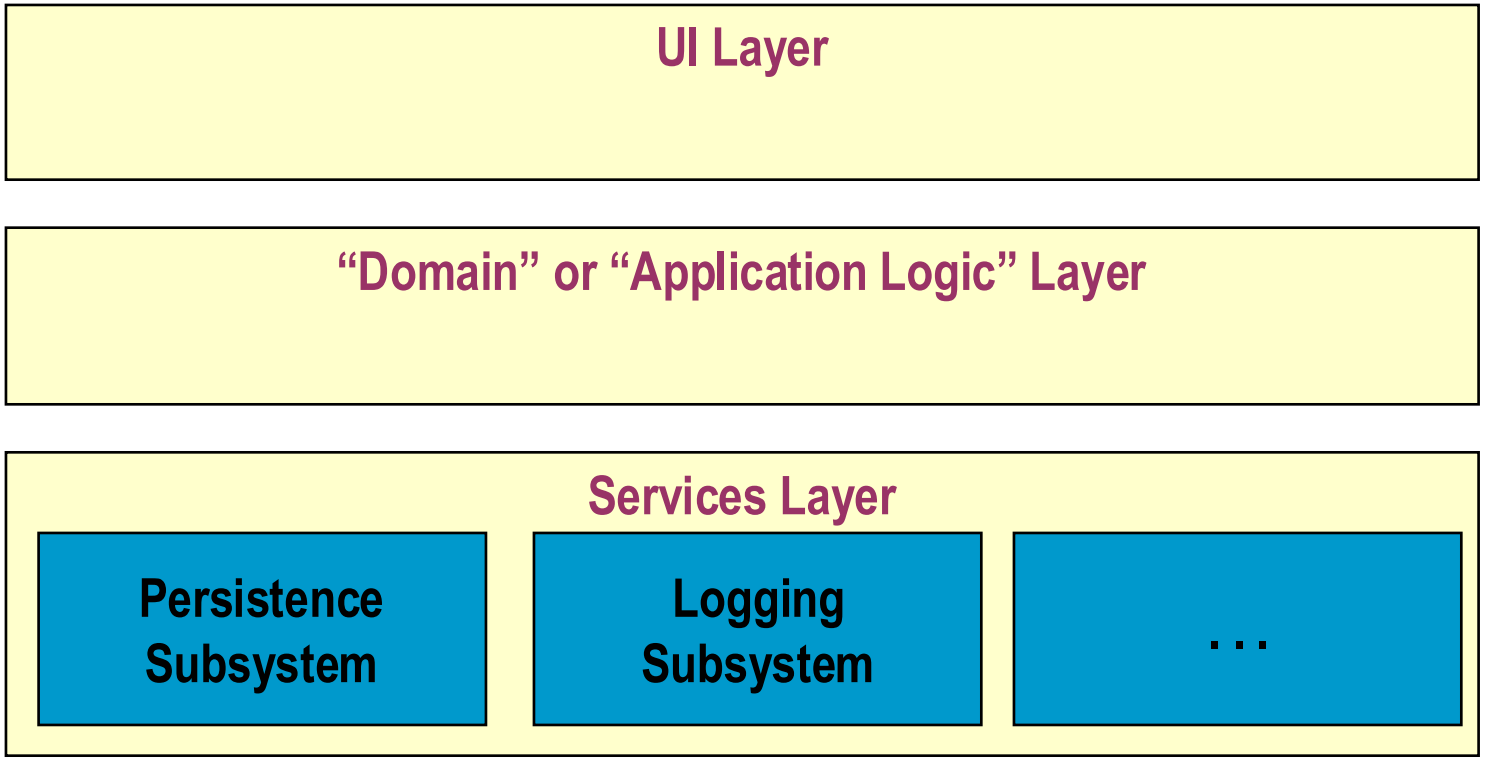
LAYERED ARCHITECTURE

Software Architecture
Introducing Patterns
Key Architecture Patterns
Model-View-Controller
Layered Architecture
Patterns and Concurrency
Conclusion



Advice: Multi-Tier Layered Architectures

- Separate presentation from application logic, and other areas of concern.



Layered Architecture

- Logical architecture organised into layers
  - Layer: coarse-grained group of classes (or subsystems or packages) with cohesive responsibility for major aspect of system.
- Relationship between layers:
  - “higher” layers call services of “lower” layers
  - Strict Layered Architecture: layer uses only layer directly below
  - Relaxed: can skip layers
- Typical layers in an OOP system:
  - User Interface
  - Application Logic + Domain Objects
  - Technical Services
    - Application-independent, reusable across systems

More Layers

- GUI windows
- reports
- speech interface
- HTML, XML, XSLT, JSP, Javascript, ...

UI
(**Presentation**, View)

- handles presentation layer requests
- workflow
- session state
- window/page transitions
- consolidation/transformation of disparate data for presentation

Application
(Workflow, Process, Mediation, Controller)

- handles application layer requests
- implementation of domain rules
- domain services (*POS*, *Inventory*) services may be used by just one application, but there is also the possibility of multi-application services

Domain
(Business, Appl Logic, Model)

- very general low-level business services used in many business domains
- E.g., *CurrencyConverter*

Business Infrastructure
(Low-level Business Services)

- (relatively) high-level technical services and frameworks
- *Persistence*, *Security*

Technical Services
(Technical Infrastructure, High-level Technical Services)

- low-level technical services, utilities, and frameworks
- *data structures, threads, maths, file, DB,*
- *and network I/O*

Foundation
(Core Services, Base Services, Low-level Technical Services/Infrastructure)

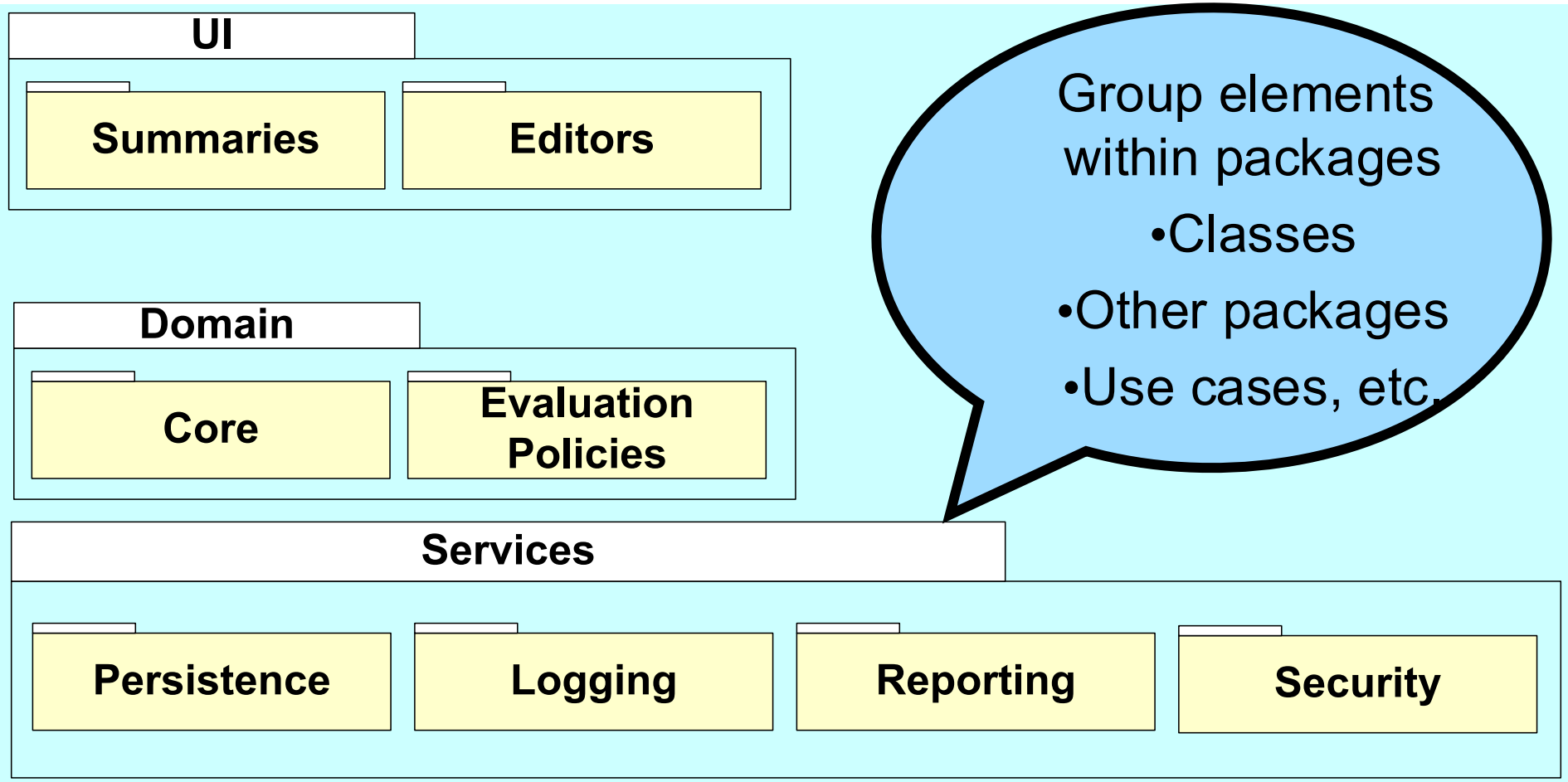
more application specific

dependency

width implies range of applicability

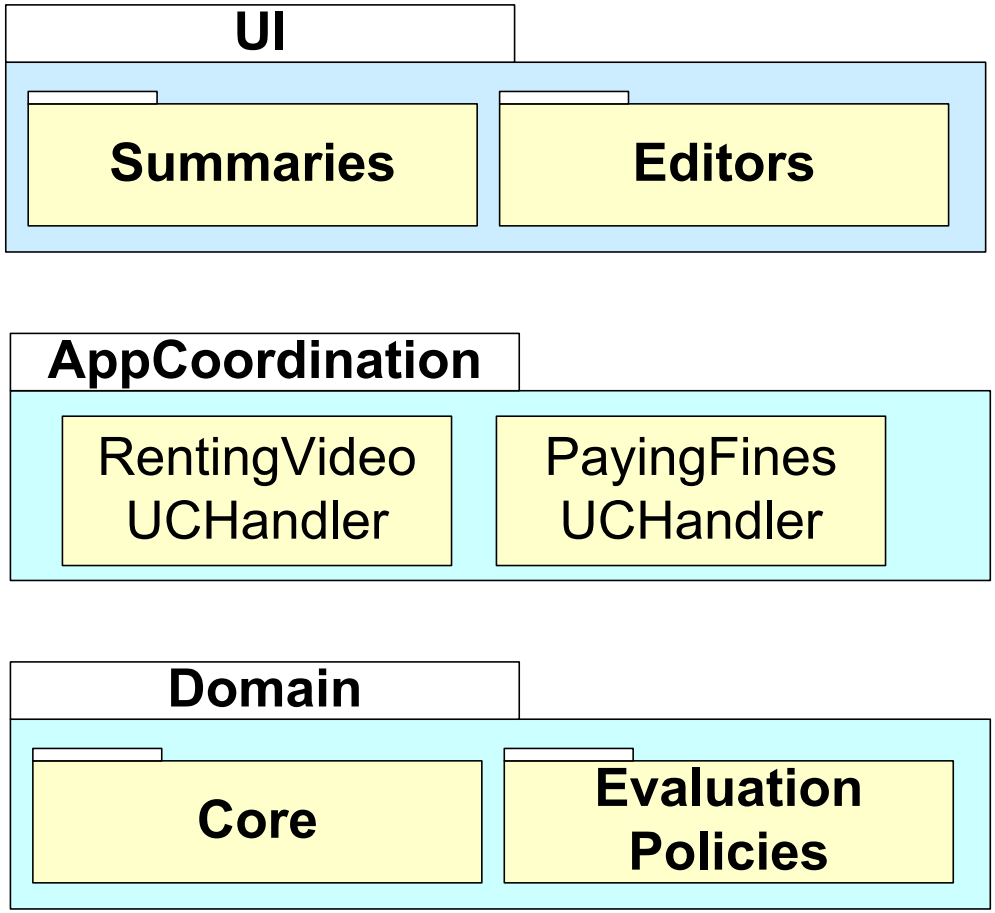
A Simple Logical Architecture

- In the UML, the logical partitioning is illustrated with package diagrams.



Advice: Application Coordination Layer

- Consider an "application coordination layer" whose objects represent use cases.
  - They may also hold session state.



Model-View Separation Principle

- Model ≡ domain layer
- View is the User Interface (UI) layer
- Model objects shouldn't have knowledge of View objects
- Don't assign domain responsibility to View objects

- Better separation of concerns with lower coupling
  - If domain layer independent of UI layer, domain layer can be used with another UI layer

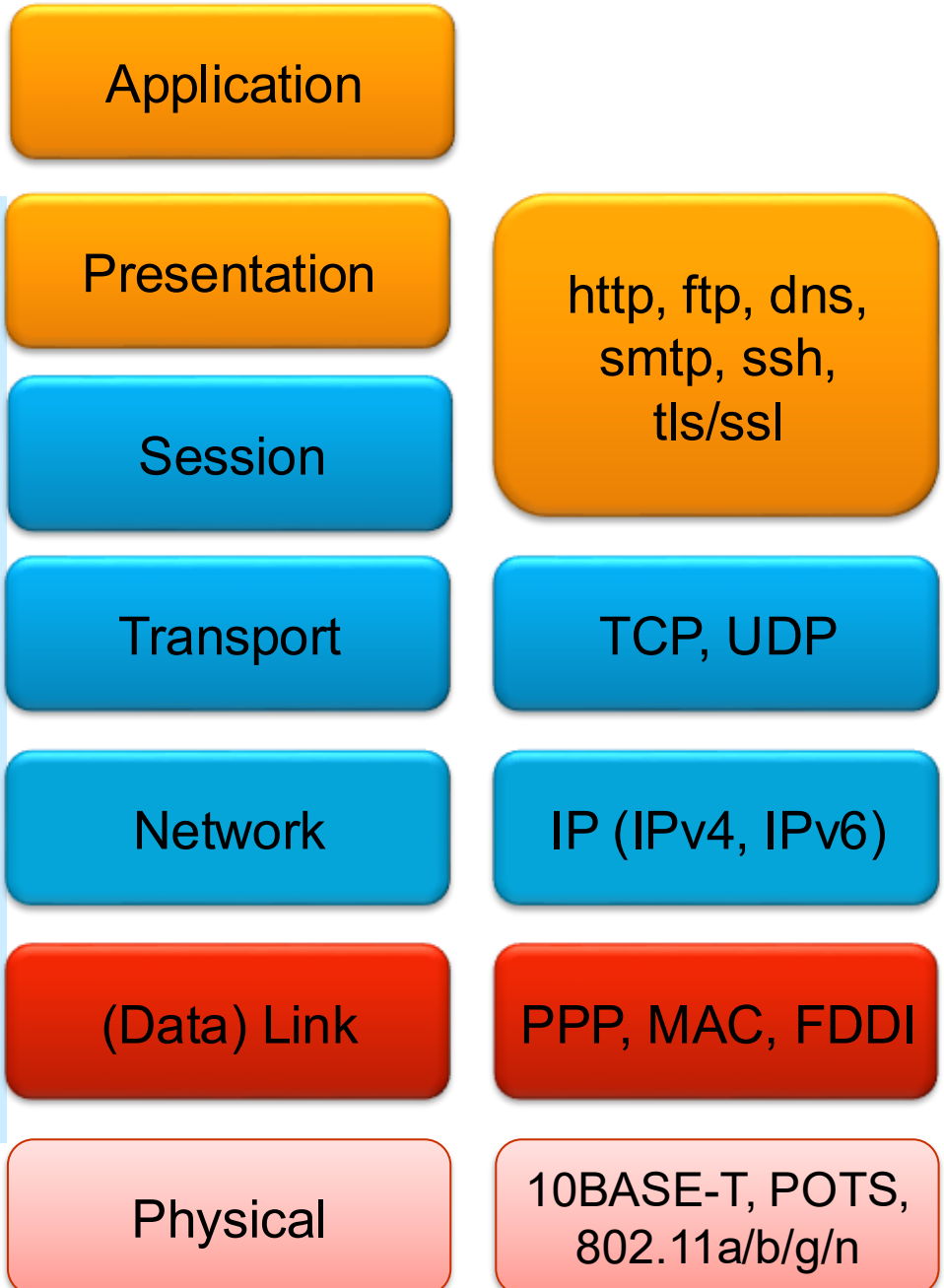
- Facilitates multiple views of a given Model
  - Example: Application and web UIs for a given application

More Benefits of Layered Architecture

- System easier to comprehend: each layer has specific purpose
- Facilitates 'high-cohesion' within, 'low-coupling' between layers
- Prevents source code changes from rippling throughout system
  - Code changes should be confined within a specific layer
- low coupling: easy to swap out a service in lower layer with another technology
  - Replacing DB without major code re-write in application logic
- Lower layers contain more reusable functionality

Internet Stack (TCP/IP Model)

rfc 1958 (1996): In searching for Internet architectural principles, we must remember that technical change is continuous in the IT industry. The Internet reflects this. Over the 25 years since the ARPANET started, various measures of the size of the Internet have increased by factors between 1000 (backbone speed) and 1000000 (number of hosts). In this environment, some architectural principles inevitably change. … *The principle of constant change is perhaps the only principle of the Internet that should survive indefinitely.*



Basic Web 2.0 Reference Architecture Diagram

Resource tier: Capabilities or backend systems (data or processing) that support services consumed over the Internet:

Service tier: packages resources as a service and controls what goes in and out.

- Application servers deploying SOAP, EJB, PHP, Rails, ASP, …

Connectivity: means to reach service

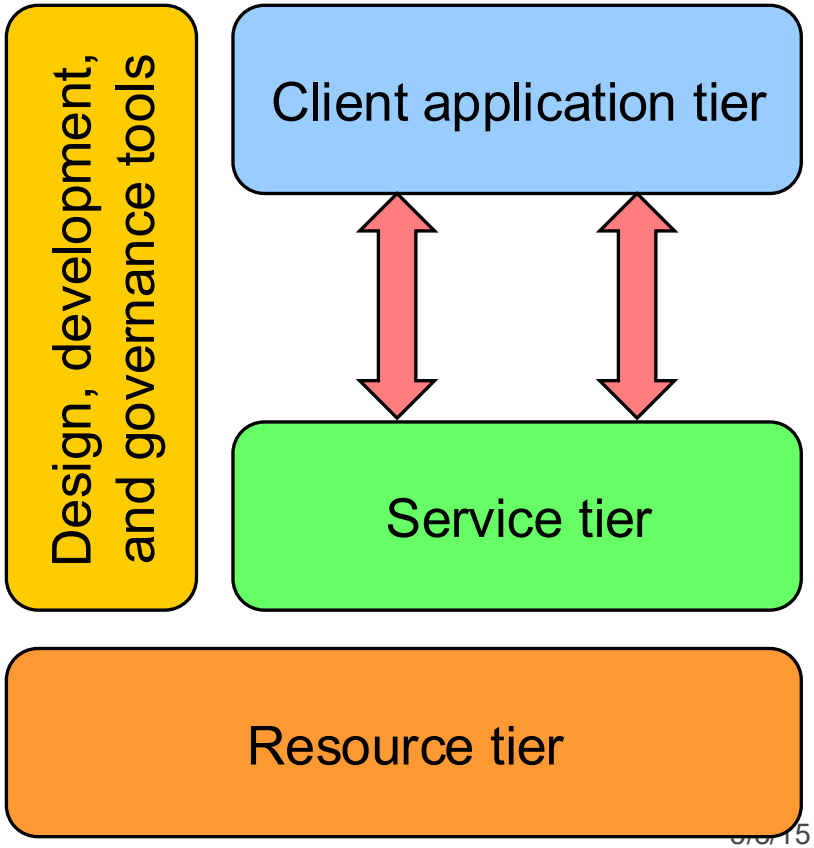
- Standards/protocols like XML over HTTP

Client tier: user view of services

- Web browsers, Flash, Acrobat, iTunes, …

Design, development, and governance tools to build web apps

- Adobe Dreamweaver and Apple's tools (xCode, DashCode), other IDEs



CONCLUSION

Software Architecture

Introducing Patterns

Key Architecture Patterns

Model-View-Controller

Layered Architecture

Patterns and Concurrency

Conclusion

Realizing non-functional requirements

Performance

- Localize critical operations and minimize communications.
- Use large rather than fine-grain components. Allow for concurrency.

Security

- Use a layered architecture with critical assets in the inner layers.

Safety

- Localize safety-critical features in a small number of sub-systems.

Availability

- Include redundant components and mechanisms for fault tolerance.
- Allow replacement of components without stopping the system.

Maintainability

- Use fine-grain, replaceable components.
- Avoid global/shared data structures.

Purpose of architecture: beforehand

- Software architecture can serve as a design plan for the negotiation of system requirements
- A means of structuring discussions with clients, developers, and managers.
- It is an essential tool for complexity management. It hides details and allows the designers to focus on the key system abstractions.

OUTLINE

Application Architectures

Data processing systems

Transaction processing systems

Information and resource management systems

Event processing systems

Language processing systems

Conclusion

Application Perspective

- So far had architectural perspectives on issues such as control, distribution and system structuring.
- Now an alternative approach: architectures from an application perspective
  - Two fundamental models of business systems
    - batch processing
    - transaction processing
  - Event processing systems
  - Structure of language processing systems

Application types

Data processing applications

- Data driven applications that process data in batches without explicit user intervention during the processing.
  - Billing systems; Payroll systems

Transaction processing applications

- Data-centred applications that process user requests and update information in a system database.
  - E-commerce systems; Reservation systems

Event processing systems

- Applications where system actions depend on interpreting events from the system's environment.
  - Games; Word processors; Real-time systems

Language processing systems

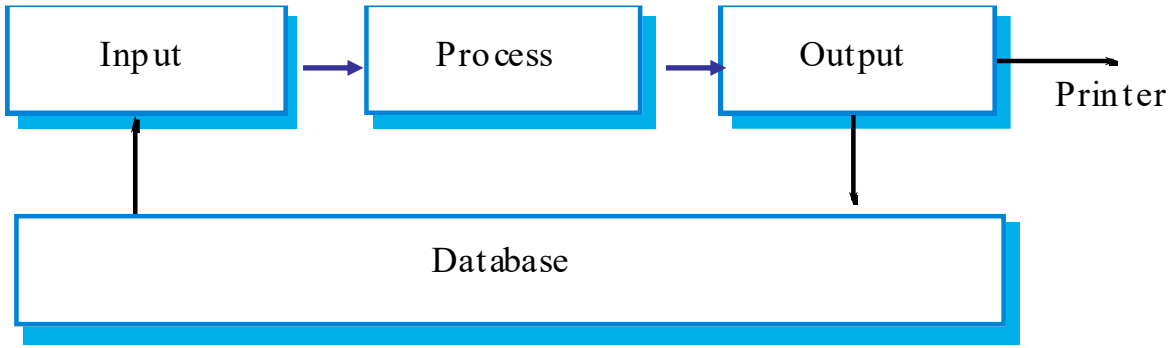
- Applications where the users' intentions are specified in a formal language that is processed and interpreted by the system.
  - Compilers; Command interpreters

Data Processing Applications - Input-process-output

input component reads data from a file or database, checks its validity and queues the valid data for processing.

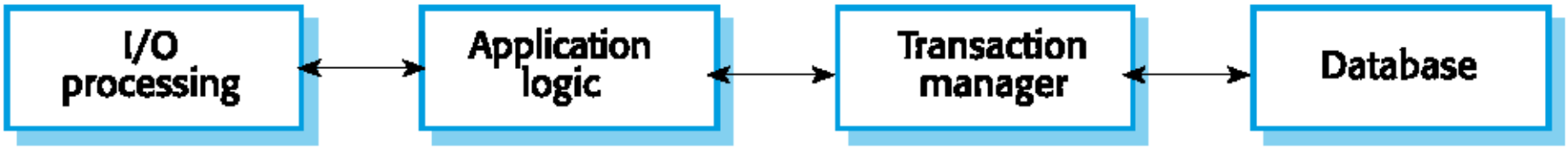
process component takes a transaction from the queue (input), performs computations and creates a new record with the results of the computation.

output component reads these records, formats them accordingly and writes them to the database or sends them to a printer.



Transaction processing systems

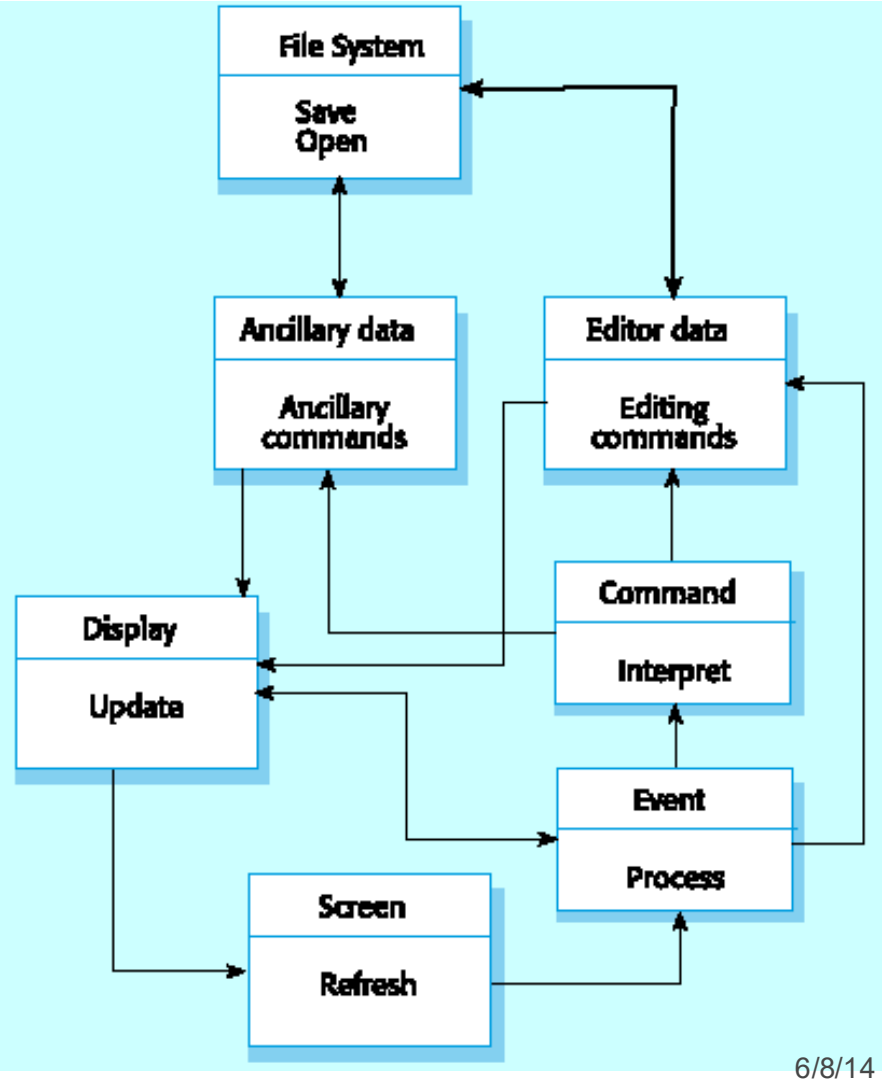
- Process user requests for information from a database or requests to update the database.
- From a user perspective a transaction is:
  - Any coherent sequence of operations that satisfies a goal;
  - For example - find the times of flights from London to Paris.
- Users make asynchronous requests for service which are then processed by a transaction manager.



Event Processing Systems - Editing system architecture

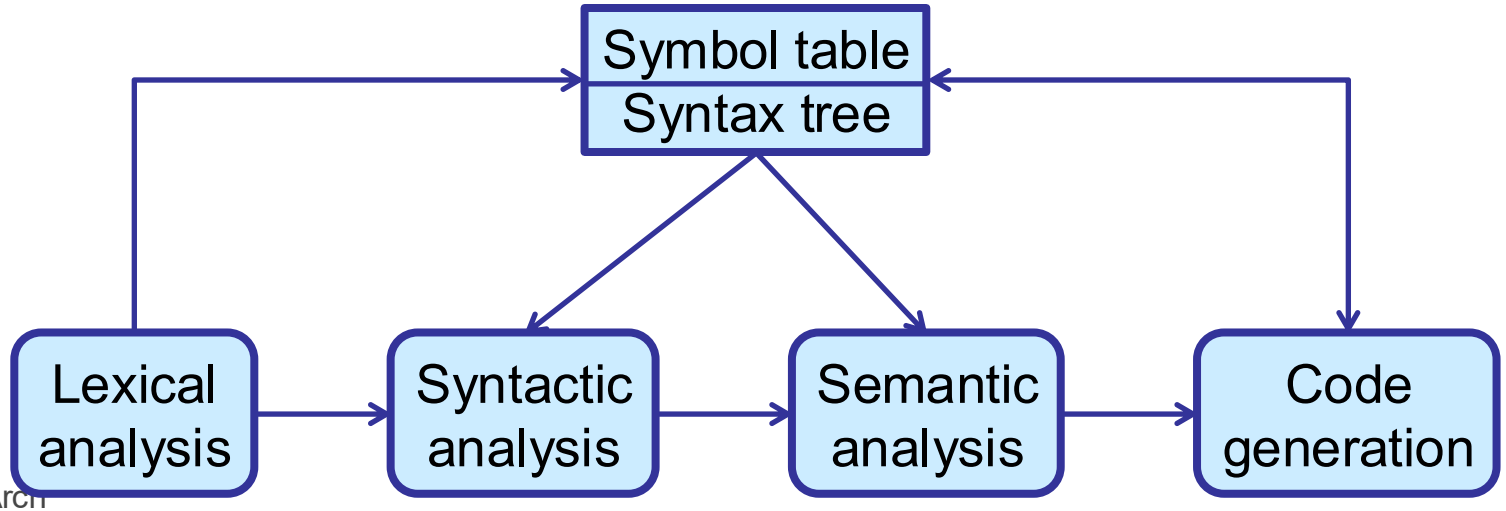
Editing systems are naturally object-oriented:

- **Screen**: monitors screen memory and detects events;
- **Event**: recognises events and passes them for processing;
- **Command**: executes a user command;
- **Editor data**: manages the editor data structure;
- **Ancillary data**: manages other data such as styles & preferences;
- **File system**: manages file I/O;
- **Display**: updates the screen display.



Compilers

- Lexical analyser
- Symbol table
- Syntax analyser
- Abstract Syntax Tree (AST)
- Semantic analyser
- Code generator



Conclusion

- Generic models of application architectures help us understand and compare applications.
- Application types are data processing systems, transaction processing systems, event processing systems and compilers.
  - or a combination
- Data processing systems operate in batch mode and have an input→process→output structure.
- Transaction processing systems allow information in a database to be remotely accessed and modified by multiple users.
- Event processing systems include editors and real-time systems.
  - In an editor, user interface events are detected and an in-store data structure is modified.
- Compilers translate source code into different language (e.g.: assembler) and may interpret the specified instructions.